

第7章 コード最適化

本章では，中間コードのレベルで意味を保ちながらより高速なコードに変換する手法について述べる．ソースコードから構文的に生成された中間コードは不要な繰り返しや変数間のコピーが多く生成されるため，制御フローとデータフローを中間コードを解析して同じ効果をもつより効率的なコードに変換する．この過程を**コード最適化 (code optimization)** と呼ぶ．さらに，最近のコンパイラで用いられている **静的単一代入形式 (Static single assignment, SSA)** を用いた最適化手法を紹介する．SSA の特徴は，その名前の通り，変数に対する代入を一度に制限して上書きしない．このことによって，変数がより多く必要になる一方，変数の名前によってデータフローを追跡することができ，基本ブロックを越えた最適化がより明確に可能になる．

7.1 基本ブロックとフローグラフ

3 番地コードレベルの抽象度を持つ言語の命令の系列に対して，以下の条件を満たす系列を**基本ブロック**という．

実行制御がその系列の先頭から最後まで，条件付きジャンプおよび無条件ジャンプは最後の場所でのみ出現し，他の命令からのジャンプ先は必ず系列の先頭しかない．

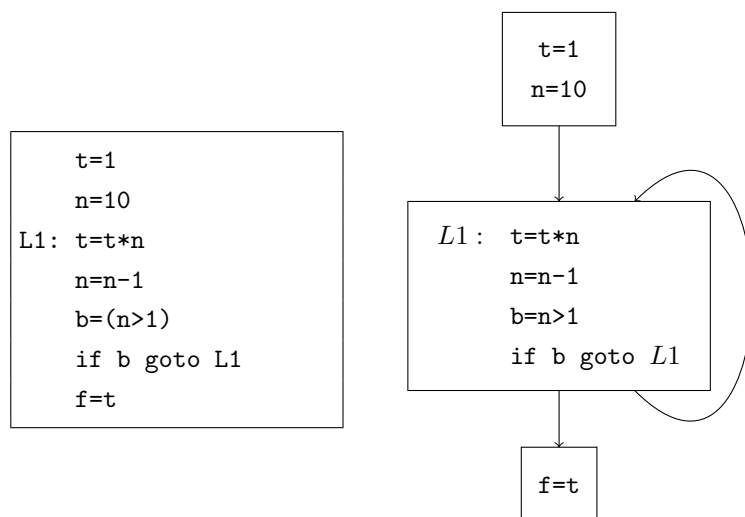


図 7.1: 3 番地コードとその CFG

基本ブロックでは，すべての命令の前提としてそれより前にある命令が必ず実行されている．3

番地コードから構成される基本ブロック内では、参照される変数について、それより前のブロック内で定義されていれば、最後に定義されたところの値を持つことがわかる。そうでなければ、先頭にジャンプしているブロック内の値が引き継がれる。

基本ブロックを頂点として、制御のフローを有向グラフで表した構造を**フローグラフ**（あるいは、**コントロールフローグラフ**、**CFG**）と呼ぶ。基本ブロック中では、制御の合流や分岐が存在しないので、CFG はプログラムの振舞いを抽象的に表現していることになる。

7.2 局所最適化

局所最適化は、3 番地コードの系列で構成される基本ブロックに対して同じ意味を持つより短くて変数の少ない系列を求める。

7.2.1 局所最適化の例

定数伝播 (constant propagation)

定数畳み込み (constant folding) とも言う。実行によって値が変わらない変数は、プログラムとして値を埋め込んでメモリ上の値の参照を減らす。

$a=2*3.14 \rightarrow a=6.28$ 定数計算をコンパイル時に行う。
 $b=a/100 \rightarrow b=0.0628$ b はコンパイル時に値が決定できる。

変数の値を参照する場合には、メモリから値を読み出す必要があるが、定数にすると値自体をコードとして生成できるので、高速になる。

共通部分式削除

同じ部分式で同じ計算の繰り返しをまとめて、計算の繰り返しを防ぐ。

$x=a*b$		$x=a*b$
$y=x+c$		$y=x+c$
$d=a*b$	$\Rightarrow$	$d=x$
$e=d-c$		$e=d-c$

以下の系列では、同じ形の部分式 $b+c$ があるが、 a と e に対する代入で値が更新されているので、 $e=a$ のような消去はできない。

$a=b+c$		$a=b+c$
$b=b-d$		$b=b-c$
$c=c+d$		$c=c+d$
$e=b+c$	$\nRightarrow$	$e=a$

コピー伝播

$a=b$ のような変数のコピーがある場合、 a と b は同じ値を保持しているので、 a を代入する必要はない。

$a=b$		$a=b$
$c=a+d$	$\Rightarrow$	$c=b+d$
$e=e-a$		$e=e-b$

このブロックの後で、 a が使われることがなければ、 $a=b$ は消去しても構わない。基本ブロックで用いられる変数については、データフロー解析などの手法を用いて求めることができる場合がある。不要コードの消去 (Dead-code elimination) によってより高速にすることができる。

代数的性質の利用

$a+0$	$\Rightarrow$	0
$b*1$	$\Rightarrow$	b
$1+(c+2)$	$\Rightarrow$	$3+c$
$d*(1+e/d)$	$\Rightarrow$	$d+e$
$x*2$	$\Rightarrow$	$x+x$
$x/5$	$\Rightarrow$	$x*0.2$

演算の単位元などによる簡単化、分配則による簡単化などによって式を短縮化する。さらに命令を低レベルのマシン語に変換するときに、除算よりも積算、積算よりも和算の方がサイクルが少ないので、より軽い命令に変更することで高速化が期待できる。

局所最適化については、個々の最適化について大きな効果は期待できない。しかし、基本ブロックがループによって繰り返し実行されるとプログラム全体では大きな効果が期待できる。

7.2.2 基本ブロックの DAG 表現

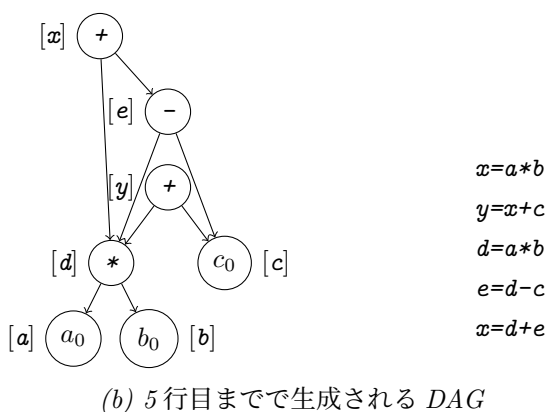
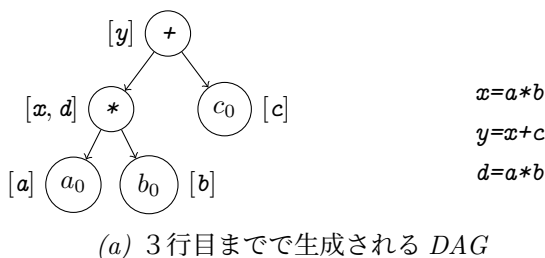
基本ブロックを有向非巡回グラフ (Directed Acyclic Graph, DAG) を用いて表すことによって、各 3 番地コードにおける参照関係を表現できる。参照関係を壊さない等価なグラフの変形によって最適化を行う。基本ブロックに対して、以下のアルゴリズムによって DAG を生成する。

アルゴリズム 7.1. DAG の頂点が始点となる辺を持たない頂点を葉頂点と呼ぶ。葉頂点には、変数または定数のラベルをつける。葉以外の頂点には、演算 θ と、演算の結果が収納される変数のラベルのリストをつける。

1. 基本ブロックに入る前に定義された値を参照する変数名と定数値に対して、葉頂点を生成し、変数 x に対して、 x_0 を頂点にラベルづけし、 $[x]$ を変数リストとする。定数 c に対しては、 c を頂点にラベル付けする。
2. 基本ブロック内のすべての代入文について上から順に以下の操作を適用する。

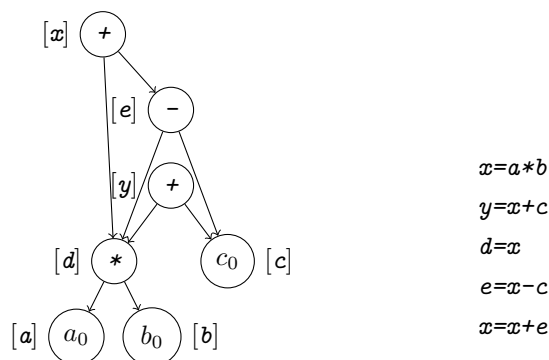
- (a) 代入文 $X = A\theta B$ に対して, A, B が変数であるときは変数ラベルに持つ頂点を v_A, v_B とする. 定数の場合は定数をラベルにもつ v_A, v_B を選ぶ.
- (b) X を変数リストに含む頂点があれば, その変数リストから X を削除する.
- (c) v_A, v_B を子に持ち, θ を演算ラベルとするノード v_θ がすでに存在するならば, v_θ の変数リストに X を追加する. v_θ が存在しないならば, v_θ を新たに生成して, v_A, v_B を子ノードとする辺を追加し, θ を v_θ の演算ラベルとし, $[X]$ を変数リストとする.
- (d) $X = A$ の場合は, すでに存在する頂点の変数リストに X が含まれれば, 消去した上で, v_A の変数リストに X を追加する.

例 7.1. 以下にアルゴリズムで生成される DAG を示す. 5 行の基本ブロックにおいて, 最初の 3 行までで, まず (a) に示す DAG が生成される.



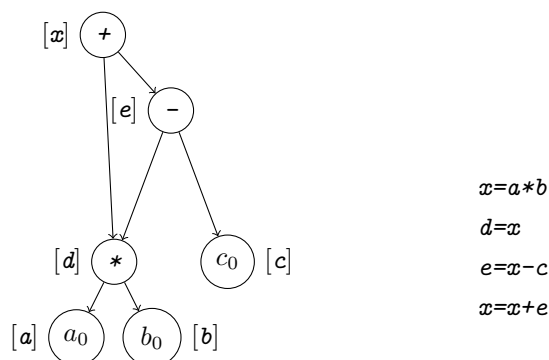
4 行目の命令でさらに e に対するノードが新たに生成され, 5 行目では x が更新されるため, 既存の変数リストから x を消去して新しく一番上のノードに x を変数リストに追加している.

例 7.1 の (a) において, $[x, d]$ のように複数の変数が一つのノードにつけられる場合は, 共通部分式が存在するということであるので共有部分式の消去が適用できる. さらに, コピー伝播を適用すると (c) のようになる.



(c) 共有部分式とコピー伝播の適用

この基本ブロックの後で y が使われない場合は, $[y]$ を変数リストにもつ頂点とその辺を消去しても意味は変わらない. これに対して, d が使われない場合は, $[d]$ の頂点を消すことはできない. これは, x が d の頂点に依存しているためである.



(d) 不要コード消去の適用

7.3 大域最適化

複数の基本ブロックにわたる最適化を大域的最適化とよぶ. この最適化を行うために, フローグラフの構造にマッチする構造を見つける方法について示す.

7.3.1 意味等価な変換

プログラムにおいて, 全体の 1, 2 割の部分が, 実際の実行の 8, 9 割の時間を占める. これは, プログラムの一部がループ構造になって繰り返し実行されることによる. 繰り返しの含まれる基本ブロックを局所最適化によって最適化することによって大きな効果が期待できることは述べた. これに加えて, ループ構造として最適化する手法について示す.

ループ構造の最適化は, 制御フローグラフに依存するため, ソースコードでも 3 番地コードでもほぼ変わらない. 以下では, C 言語プログラムの一部で最適化の手法について示す.

ループ最適化

ループ不変コードの移動 ループ内で値が変化しない変数をループの外に移動し、計算に影響しない命令が何度も実行されずに同じ意味を表現するようにする。

```

i=1;
do {
    c=a/b;x=i*c; .... i++;
} while(i<n);
⇒
i=1;t=a/b;
do {
    c=t;x=i*c;...i++;
} while(i<n);

```

do ... while で、a, b のいずれも更新されないとき、c はループ実行で変化しないため、ループ外で値を一度代入する同じ計算となる。

```

for(i=1;i<n;i++) {
    c=a/b;x=i*c;...
}
⇒
if(n>1) {
    t=a/b;
    for(i=1;i<n;i++) {
        c=t;x=i*c;...
    }
}

```

一般的な for ループでは、条件によってループの内容が実行されないこともあるため、不変コードはループが実行された場合だけ、実行されるように変換する必要がある。

ループ内演算の強さ軽減

```

for(i=1;i<n;i++)
    a=a+b*i;
⇒
t=0;
for(i=1;i<n;i++) {
    t=t+b;a=a+t;
}

```

帰納変数の消去

```

t=b;
for(i=1;i<n;i++) {
    a=a+t;t=t+b;
}
⇒
m=n*b;
for(t=b;t<m;t=t+b)
    a=a+t;

```

ループ融合

```

for(i=1;i<n;i++)
    a[i+1]=a[i]+i;
for(i=1;i<n;i++)
    b[i]=a[i]+i;
⇒
for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+i;
}

```

ループ分解

```

for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+i;
}
⇒
for(i=1;i<n;i++)
    a[i+1]=a[i]+i;
for(i=1;i<n;i++)
    b[i]=a[i]+i;

```

ループ展開

```

for(i=1;i<n;i++) {
    a[i+1]=a[i]+i;
    b[i]=a[i]+b[i-1];
}
⇒
for(i=1;i<n;i++) {
    a[i]=a[i-1]+i;
    b[i]=a[i]+b[i-1];a[i+1]=a[i]+1;
    b[i+1]=a[i+1]+b[i];
}

```

ソフトウェアパイプライン

```

for(i=1;i<1000;i++) {
    a[i]=a[i-1]+1; // Ai
    b[i]=a[i]+b[i-1]; // Bi⇒
    c[i]=b[i]+2; // Ci
}
A1;
B1; A2;
for(i=3;i<1000;i++) {
    Ci-2;Bi-1;Ai;
}
C998;B999;
C999;

```

手続き呼び出し最適化

インライン展開 手続き呼び出しは、スタック上に局所環境を作成し、リターンの際にもスタック上の局所環境をクリアする必要がある。局所環境を作成するほどでもない単純な手続き呼び出しでは、スタック操作を行わなくてもよい場合もあり、ループ構造内では大きな高速化の効果が期待できる。

```

int square(int x) {
    return x*x;
}
⇒ y = a * a;
y = square(a);

```

c 言語では、関数の前に `inline` を付加することでスタックを用いずに呼び出しを行う。ただし、`inline` をつけた場合は、実際にインライン展開するかどうかはコンパイラで判断する。必ず展開する場合は、`#define` でマクロ定義すれば呼び出しと同様の形式でプログラムを記述できる。

末尾再帰 再帰呼び出しは、呼び出し定義の中で自分を呼び出すことである。定義の中の呼び出しが定義の最後の命令であるとき、関数からリターンしたあともとの環境の変数は参照されないのので、環境を復元する必要がない。従って、インラインでない関数呼び出しを行う場合であっても、毎回、局所環境を作成する必要がなく、最初の1回目の呼び出しで作成した領域を上書きすればよい。変換としては、再帰呼び出しを `goto` に置き換えることで高速化が可能である。

```

int sum_tail(int n, int acc) {
    if (n == 0) {
        return acc;
    }
    return sum_tail(n-1,acc+n);
}
⇒
int sum_tail(int n, int acc) {
L:  if (n == 0) {
        return acc;
    }
    acc+=n; n--;
    goto L;
}

```

goto によるループ構造を goto なしで記述すると以下のプログラムと等価である.

```
int sum(int n,int acc) {  
    while(n!=0) {  
        acc+=n; n--;  
    }  
    return acc;  
}
```

7.3.2 データフロー解析

到達定義解析

生存解析

利用可能式解析

7.4 静的単一代入形式と最適化

最近のコンパイラにおける最適化では, 静的単一代入形式 (Static single assignment, SSA) を用いる事が多い. SSA 形式では中間コード列の中に各変数の定義 (3 番地コードの場合は代入の左辺) が一箇所しか現れないという制約を持つ. 定義が 1 つしかないため, どこで定義された変数がどこで参照されるかが, プログラムとして明示される. プログラムを解析すれば, プログラムの構文情報からデータフローを解析することができる.

7.4.1 例

7.4.2 ϕ 関数

7.4.3 SSA 形式への変換

7.4.4 SSA 上での最適化

7.4.5 逆変換